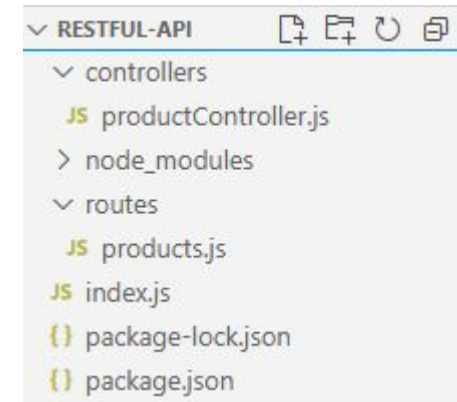


Pengantar Projek

- Projek yang akan dibuat adalah backend Application yang dapat dijalankan lewat aplikasi yang dapat men-consume API seperti Postman dan semacamnya.
- Untuk penjelasan RESTful API dapat dilihat pada unit kompetensi yang lain
- Untuk struktur folder dan file, tidak ada keharusan mutlak dari Express.js sendiri. Express sangat fleksible, kita dapat Meletakkan semua kode dalam 1 file (index.js) atau memecah jadi folder seperti routes/, controllers/, models/, dll. Tetapi,
- Struktur folder yang rapi sangat penting jika proyek semakin besar, bekerja dalam tim lain, ingin menerapkan prinsip *separation of concern*

Langkah-langkah membuat aplikasi

1. Buat projek node baru dengan nama restful-api
2. Inisial projek dan install express.js
3. Diluar file dan folder yang terbentuk, buatlah beberapa file dan folder seperti :
 - **index.js** -> Entry point
 - **routes/**
 - **products.js** -> Routing RESTfull
 - **controllers/**
 - **productController.js** -> Logic Pemrosesan



Langkah-langkah membuat aplikasi

4. Pada index.js tambahkan script di bawah ini :

```
const express = require('express');
const app = express();
const productRoutes = require('./routes/products');

app.use(express.json()); // Middleware parsing JSON
app.use('/products', productRoutes); // Prefix endpoint

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server RESTful berjalan di http://localhost:${PORT}`);
});
```

Langkah-langkah membuat aplikasi

5. Pada products.js tambahkan script di bawah ini :

```
const express = require('express');
const router = express.Router();
const controller = require('../controllers/productController');

// Routing RESTful
router.get('/', controller.getAllProducts);
router.get('/:id', controller.getProductById);
router.post('/', controller.createProduct);
router.put('/:id', controller.updateProduct);
router.delete('/:id', controller.deleteProduct);

module.exports = router;
```

Langkah-langkah membuat aplikasi

6. Pada **controllers/productController** tambahkan script di bawah ini, untuk membuatkan inisialisasi awal variabel products dalam bentuk array :

```
let products = [  
  { id: 1, name: 'Produk A', price: 10000 },  
  { id: 2, name: 'Produk B', price: 15000 }  
];
```

Langkah-langkah membuat aplikasi

7. Masih **controllers/productController** tambahkan script di bawah ini, untuk membuat logic mendapatkan seluruh product (getAllProducts):

```
exports.getAllProducts = (req, res) => {  
  res.status(200).json(products);  
};
```

Langkah-langkah membuat aplikasi

8. Masih **controllers/productController** tambahkan script di bawah ini, untuk membuat logic mendapatkan product by Id(getProductById):

```
exports.getProductById = (req, res) => {  
  const id = parseInt(req.params.id);  
  const product = products.find(p => p.id === id);  
  if (!product) return res.status(404).json({ message: 'Produk tidak ditemukan' });  
  res.status(200).json(product);  
};
```

Langkah-langkah membuat aplikasi

9. Masih **controllers/productController** tambahkan script di bawah ini, untuk membuat logic Create produk baru(createProduct):

```
exports.createProduct = (req, res) => {  
  const { name, price } = req.body;  
  if (!name || typeof price !== 'number') {  
    return res.status(400).json({ message: 'Input tidak valid' });  
  }  
  
  const newProduct = {  
    id: products.length ? Math.max(...products.map(p => p.id)) + 1 : 1,  
    name,  
    price  
  };  
  
  products.push(newProduct);  
  res.status(201).json(newProduct);  
};
```

Langkah-langkah membuat aplikasi

10. Masih **controllers/productController** tambahkan script di bawah ini, untuk membuat logic update product (updateProduct):

```
exports.updateProduct = (req, res) => {  
  const id = parseInt(req.params.id);  
  const product = products.find(p => p.id === id);  
  if (!product) return res.status(404).json({ message: 'Produk tidak ditemukan' });  
  
  const { name, price } = req.body;  
  if (!name || typeof price !== 'number') {  
    return res.status(400).json({ message: 'Input tidak valid' });  
  }  
  
  product.name = name;  
  product.price = price;  
  res.status(200).json(product);  
};
```

Langkah-langkah membuat aplikasi

11. Masih **controllers/productController** tambahkan script di bawah ini, untuk membuat logic menghapus produk (deleteProduct):

```
exports.deleteProduct = (req, res) => {  
  const id = parseInt(req.params.id);  
  const index = products.findIndex(p => p.id === id);  
  if (index === -1) return res.status(404).json({ message: 'Produk tidak ditemukan' });  
  
  products.splice(index, 1);  
  res.status(204).send();  
};
```

Langkah-langkah membuat aplikasi

12. Jalankan Server dengan mengetikkan **node index.js**:

```
PS D:\> cd "C:\Users\user\Documents\restful-api" & node index.js
Server RESTful berjalan di http://localhost:3000
```

13. Test aplikasi dengan aplikasi **Postman** atau extension **Thunder Client** di **Vs.Code**
14. Untuk menjalankannya url yang digunakan adalah `http://localhost:3000` + Endpoint RESTfull seperti pada list di bawah:

Method	Endpoint	Keterangan
GET	/products	Ambil semua produk
GET	/products/:id	Ambil satu produk
POST	/products	Tambah produk baru
PUT	/products/:id	Update produk tertentu
DELETE	/products/:id	Hapus produk tertentu

Langkah-langkah membuat aplikasi

15. Sebagai contoh untuk mengakses semua produk, maka urlnya adalah :

The screenshot displays a REST client interface with the following components:

- Method:** A dropdown menu showing "GET".
- Endpoint:** A text input field containing "http://localhost:3000/products".
- Buttons:** A "Send" button and a "Klik" label.
- Tabs:** "Query", "Headers", "Auth", "Body", "Tests", and "Pre Run".
- Query Parameters:** A section with a checkbox labeled "parameter" and a "value" field.
- Status:** "Status: 200 OK", "Size: 83 Bytes", and "Time: 11 ms".
- Response:** A JSON array of two product objects.

Red annotations highlight the "Method", "Endpoint", and "Response" sections. The response is a JSON array containing two product objects:

```
1  [  
2    {  
3      "id": 1,  
4      "name": "Produk A",  
5      "price": 10000  
6    },  
7    {  
8      "id": 2,  
9      "name": "Produk B",  
10     "price": 15000  
11   }  
12  ]
```

The response is labeled "Hasil / Response" in red.

Langkah-langkah membuat aplikasi

16. Untuk mengakses produk by id, maka urlnya adalah :

GET ⌵ http://localhost:3000/products/1 Send		Status: 200 OK Size: 40 Bytes Time: 4 ms	
<u>Query</u> Headers ² Auth Body Tests Pre Run		<u>Response</u> Headers ⁶ Cookies Results Docs	
Query Parameters		<pre>1 { 2 "id": 1, 3 "name": "Produk A", 4 "price": 10000 5 }</pre>	
☐	parameter		value

Langkah-langkah membuat aplikasi

17. Untuk mengakses createProduk, maka urlnya adalah :

The screenshot displays a REST client interface with the following components:

- Method and URL:** The method is set to **POST** (labeled 1) and the URL is **http://localhost:3000/products** (labeled 2).
- Body:** The **Body** tab (labeled 1) is selected, showing **JSON** content (labeled 4). The JSON payload is:

```
1 {
2   "name" : "Produk Baru",
3   "price" : 15000
4 }
```

 (labeled 5).
- Send Button:** A blue **Send** button (labeled 6) is located to the right of the URL.
- Status and Metadata:** The status is **201 Created**, size is **43 Bytes**, and time is **4 ms**.
- Response:** The **Response** tab (labeled 6) shows the returned JSON:

```
1 {
2   "id": 3,
3   "name": "Produk Baru",
4   "price": 15000
5 }
```